
Lab 2 – Formal Verification with JasperGold

Lab 2 – COMPENG 4DV4

*Chris Hoftyzer 400410904 - Group
12*

Checkpoint 1:	2
Checkpoint 2:	3
Checkpoint 3:	4
Checkpoint 4:	5

Initially the coverage condition `cov_empty_to_full_to_empty` was unreachable. This condition failed because of the line:

`empty ##1 write_en ##1 full ##1 read_en ##1 empty`

This line states a condition that the full signal will be high 1 clock cycle after asserting `write_en` after an empty, and afterwards the empty signal will be high after asserting `read_en`. Because the FIFO has a depth of 8, this will not be the case. To get a full signal following an empty signal, we must perform 8 writes, and to get an empty signal following an empty signal, we must perform 8 reads. This line should be rewritten in the following way:

`empty ##1 write_en[->8] ##1 full ##1 read_en[->8] ##1 empty`

Checkpoint 1:

After updating our `wr_ptr` update logic, our assertion `ast_write_clears_empty` fails. This is because our new (incorrect) logic does not update the `wr_ptr` when writing to the fifo. When the FIFO is initially empty, the empty signal will stay high indefinitely because the `wr_ptr` is not updated after writes

The screenshot shows the Cadence Formal Property Verifier interface. The Design Hierarchy on the left shows a project named 'fifo formal (fifo formal)' with a sub-component 'dut (fifo)'. The Property Table on the right lists various properties. The assertion 'fifo_formal.ast_write_clears_empty' is highlighted in blue and marked with a red 'X' icon, indicating it has failed. The table shows the following data for this assertion:

Type	Name	Engine	Bound	Traces	Time
Assume	fifo_formal.assume_no_read_empty	?		0	
Cover (related)	fifo_formal.assume_no_read_empty:precondi...	PRE	1	1	
Assume	fifo_formal.assume_no_write_full	?		0	
Cover (related)	fifo_formal.assume_no_write_full:precondition1	PRE	Infinite	0	
Assume	fifo_formal.assume_no_simul_rw	?		0	
Assert	fifo_formal.ast_remain_full_until_read	PRE	Infinite	0	
Cover (related)	fifo_formal.ast_remain_full_until_read:precon...	PRE	Infinite	0	
Assert	fifo_formal.ast_remain_empty_until_write	PRE	Infinite	0	
Cover (related)	fifo_formal.ast_remain_empty_until_write:pre...	PRE	1	1	
Assert	fifo_formal.ast_full_empty_mutex	PRE	Infinite	0	
Assert	fifo_formal.ast_read_clears_full	PRE	Infinite	0	
Cover (related)	fifo_formal.ast_read_clears_full:precondition1	PRE	Infinite	0	
Assert	fifo_formal.ast_write_clears_empty	PRE	2	1	
Cover (related)	fifo_formal.ast_write_clears_empty:precondit...	PRE	1	1	
Cover	fifo_formal.cov_reach_full	PRE	Infinite	0	
Cover	fifo_formal.cov_empty_to_full_to_empty	PRE	Infinite	0	

The bottom status bar shows 'No proofs running' and 'Console input ready'.

Figure 1 – Failed assertion `ast_write_clears_empty` due to updating `wr_ptr`

Checkpoint 2:

Double clicking the failed assertion opens the waveform. The waveform shows that on clock cycle 1, empty and write_en are high. Our assertion ast_write_clears_empty requires that empty transition to low the clock cycle after a write, but on clock cycle 2, empty is still high, contradicting our assertion.

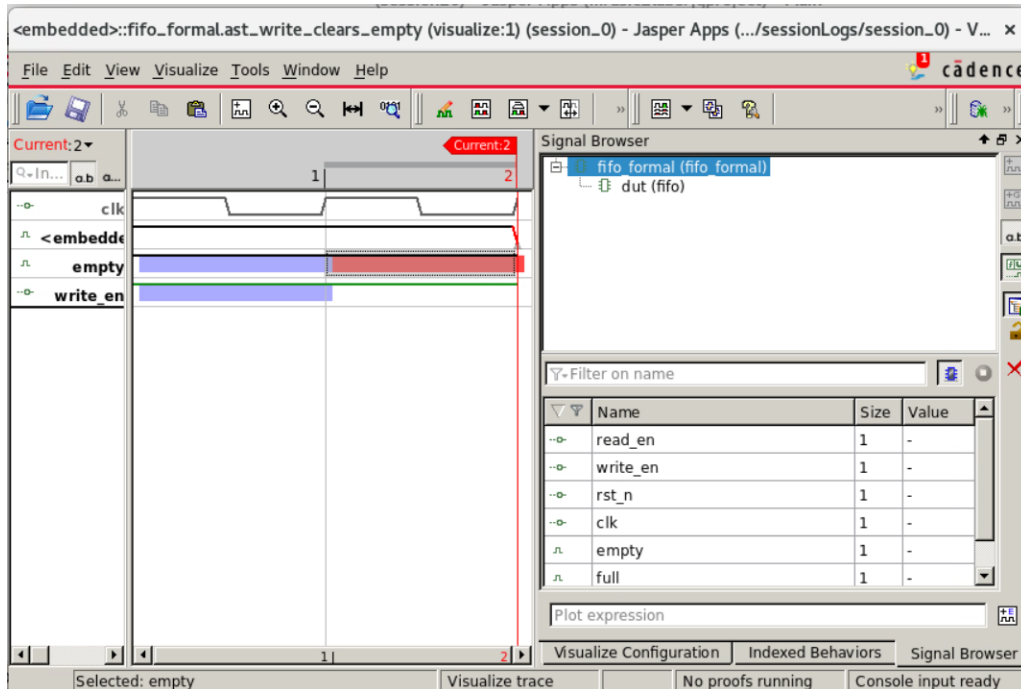


Figure 2 – waveform of failing ast_write_clears_empty assertion

Checkpoint 3:

Opening why analysis shows the assignment of empty in our RTL code. Empty is high when rd_ptr and wr_ptr are equal. Opening why analysis for wr_ptr and rd_ptr show how they are updated/assigned. Why analysis shows that wr_ptr is not incremented after a write and that it is initialized to 4'b0000 out of reset. The assertion fails because wr_ptr and rd_ptr are both initialized to 4'b0000 out of reset. When a write occurs, the FIFO is no longer empty, but since wr_ptr is not updated, the empty signal remains high and the assertion fails.

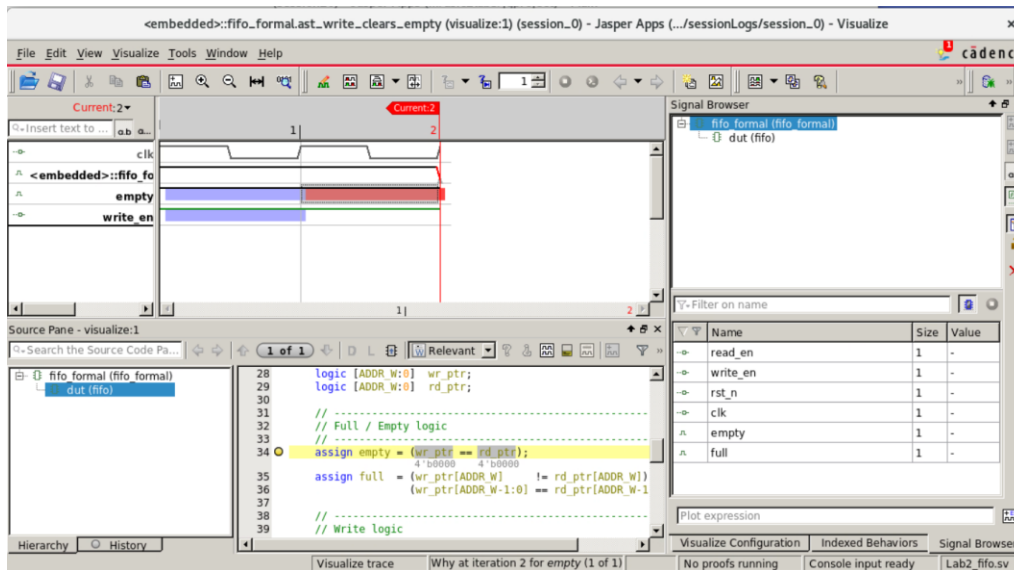


Figure 3 – Why debug showing RTL assignment of empty output

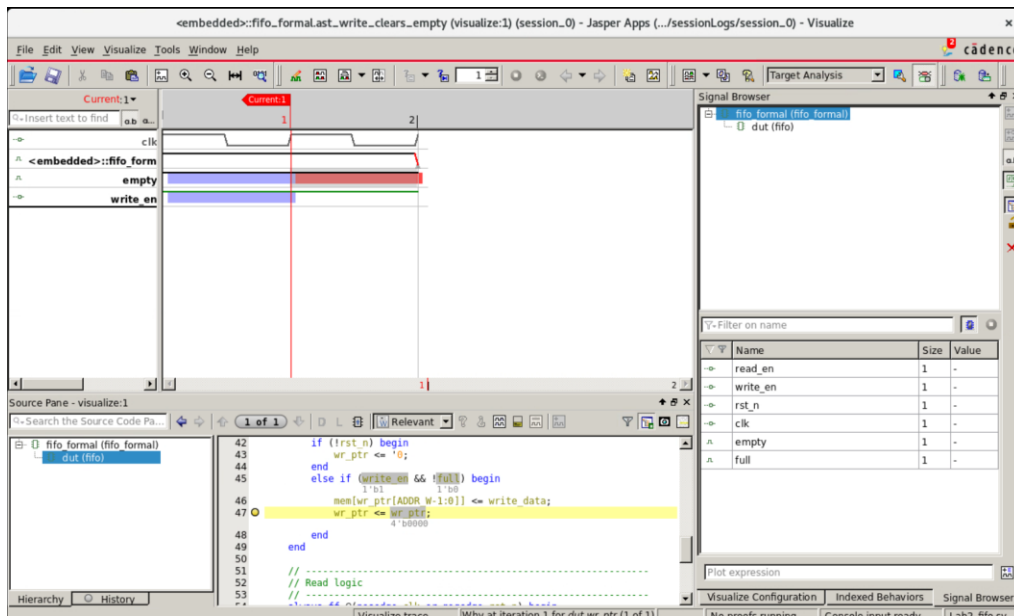


Figure 4 – Why debug showing update logic for wr_ptr

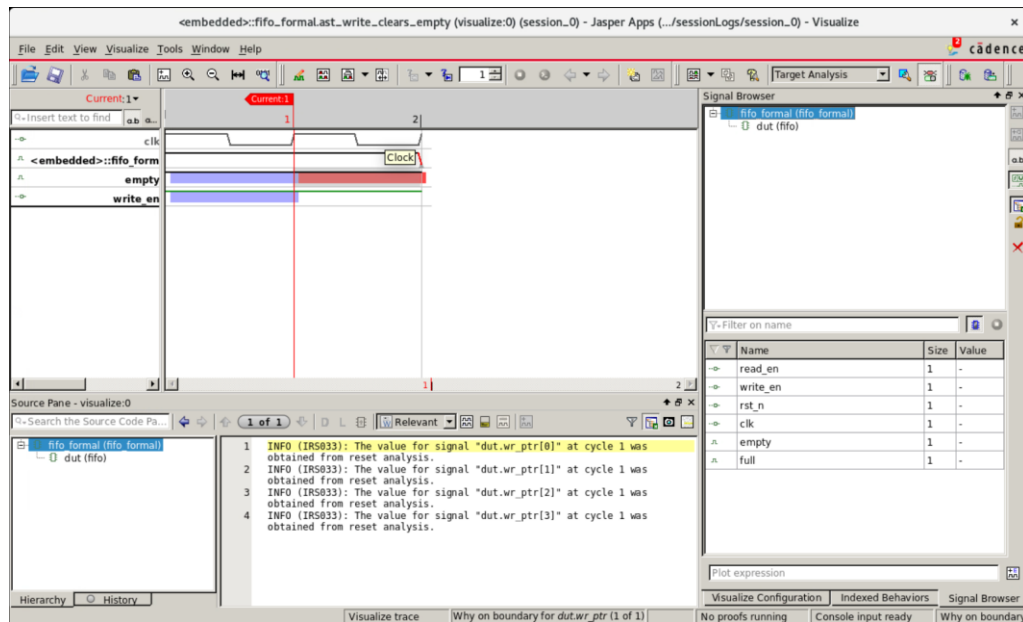


Figure 5 – Why debug showing reset update logic for wr_ptr

Checkpoint 4:

Selecting highlight relevant inputs highlights write_en and read_en on the preceding clock cycle. These signals are relevant inputs because they are used to as select signals for a mux to determine if rd_ptr and wr_ptr are updated. Highlighting relevant inputs helps the RTL designer narrow down what the source of any logic errors may be.

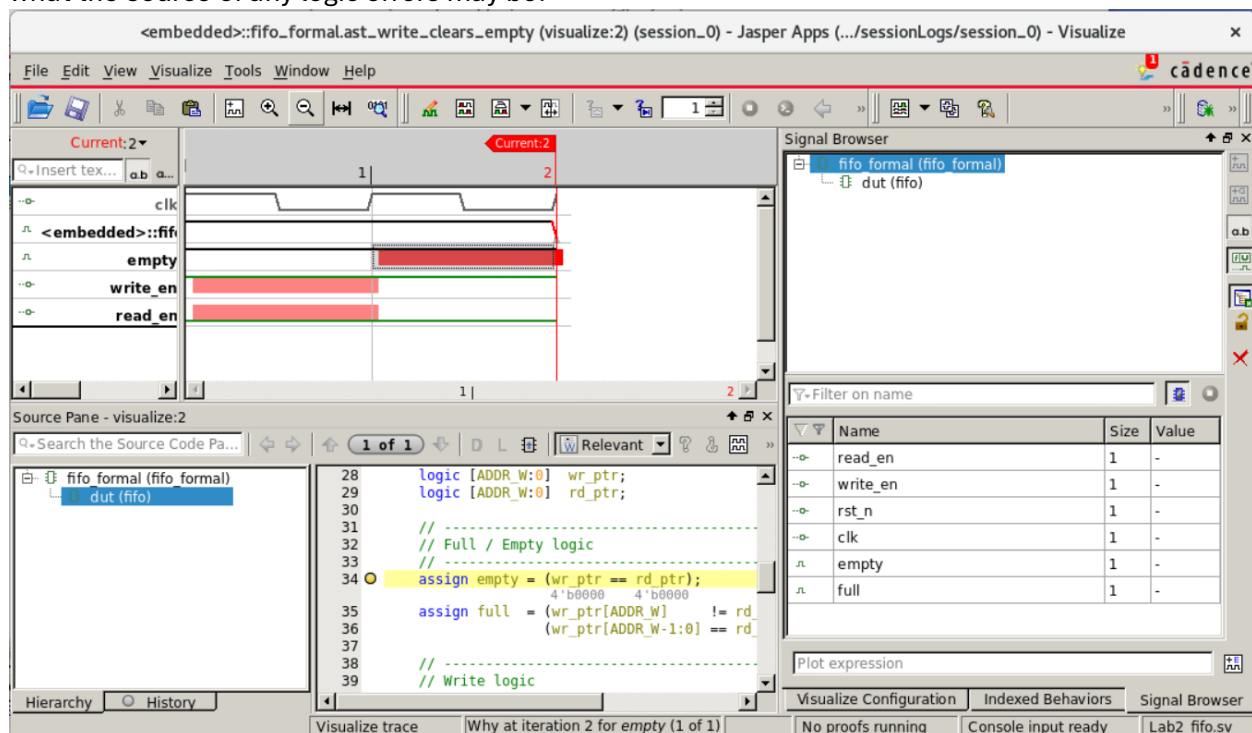


Figure 6 – Why debug with relevant inputs highlighted